AI/ML Track

Interview Preparation Series • Professional Edition

MLOps & ML System Design

06

Compiled with LaTeX • June 15, 2026

harsha.somisetty01@gmail.com

In This Edition

1 MLOps & ML System Design

> MLOps is the engineering discipline that closes the loop between ML research and production impact — the difference between a model that scores well in a notebook and one that earns billions of dollars reliably at scale.

1.1 Overview --- What it is

MLOps (Machine Learning Operations) is the set of practices, tools, and cultural norms that make it possible to deploy, monitor, and continuously improve ML systems in production. ML System Design is the interview skill of architecting those systems end-to-end given business and technical constraints.

MLOps spans five lifecycle phases:

1. **Data** — ingestion, validation, versioning, labeling
2. **Features** — engineering, storing, serving (online + offline)
3. **Training** — distributed computation, experiment tracking, hyperparameter search
4. **Deployment** — packaging, serving, traffic management
5. **Monitoring & Feedback** — drift detection, alerting, triggering retraining

ML System Design adds the business layer:

- Translate a vague product goal (”make users engage more”) into a precise ML objective
- Choose the right model architecture, feature set, and serving strategy
- Reason about latency, cost, fairness, and iteration speed from the start

1.2 Why It Exists

Classical software engineering has a clean input-output contract: given the same code and inputs, outputs are deterministic and bugs are reproducible. ML systems break every assumption:

Classical Software	ML Systems
Logic is explicit in code	Logic is latent in model weights
Bug = wrong code	Bug = wrong data, wrong objective, or distribution shift
Version = git commit	Version = code + data + hyperparameters + environment
Testing = unit tests	Testing = statistical evaluation on held-out sets
Deployment is one-time	Deployment requires continuous retraining
Failure is deterministic	Failure is gradual (silent model decay)

MLOps exists because the ML development lifecycle is fundamentally different from software development. Without it, teams experience:

- **Training-serving skew**: model trained on offline data that doesn't match production
- **Silent degradation**: model accuracy drifts without triggering any alert
- **Unreproducible experiments**: ”which run produced the model we shipped?”
- **Deployment bottlenecks**: data scientists can't deploy; engineers can't understand models

1.3 Why FAANG Cares (be specific --- recsys/ranking/ads are the revenue engines)

At FAANG scale, ML systems are not infrastructure — they ARE the product.

The numbers that matter:

Company	ML System	Revenue / Business Impact
Google	Ad CTR prediction	$220B + /yradrevenue; 12B
Meta	News Feed ranking	Core DAU driver; feed determines time-on-site
Amazon	Product recommendations	35% of Amazon's revenue attributed to recommendations
Netflix	Content recommendations	Saves ~$1B/yr in churn; 80% of watched content is recommended
TikTok	For You page ranking	Primary retention mechanism for 1B+ MAU

Why FAANG interviews specifically test this:

- Senior engineers at FAANG own systems generating millions of predictions per second
- Model quality improvements translate directly to revenue (each 0.1% AUC lift in ads = $100M+)
- MLOps failures are catastrophic: a bad model pushed to 100% traffic can crash a product in hours
- System design interviews test whether you can reason about scale before writing a single line of code

The FAANG-specific challenges:

- **Scale**: billions of users, trillions of features, millions of QPS
- **Freshness**: news feed needs features from the last 30 seconds
- **Fairness & Safety**: models making consequential decisions (loans, content moderation)
- **Multi-objective optimization**: engagement vs. revenue vs. diversity vs. user wellbeing

1.4 The ML System Design Framework (Step-by-Step Playbook)

Use this framework in every ML system design interview. Walk through each step explicitly — interviewers reward structured thinking.

```
FRAMEWORK: C-O-D-F-M-T-E-S-M-I
Clarify → Objective → Data → Features → Model → Train → Evaluate → Serve → Monitor → Iterate
```

Step 1: Clarify Requirements (5 min)

Always start here. Never design before clarifying.

Questions to ask:

- **Scale**: How many users? How many items/candidates? QPS at serving time?
- **Latency**: Real-time (< 100ms)? Near-real-time (< 1s)? Batch (hours)?
- **Freshness**: How stale can features/model be? (seconds? hours? days?)
- **Constraints**: Budget? Existing infrastructure? Privacy (GDPR, CCPA)?
- **Cold start**: What happens with new users or new items?
- **Feedback**: Explicit (ratings) or implicit (clicks, dwell time)?

Interview tip: Say ”Before I design, let me ask a few clarifying questions to make sure I'm solving the right problem.”

Step 2: Define ML Objective & Metrics

Translate the business goal into a precise ML problem.

Business Goal	ML Objective	Primary Metric	Business Metric
Increase engagement	Maximize click probability	AUC-ROC, log-loss	CTR, DAU
Increase revenue	Maximize conversion × revenue	NDCG, MAP	Revenue/query
Reduce churn	Predict 30-day churn	F1, Recall	Retention rate
Improve search	Rank relevant docs higher	NDCG@10, MRR	Click-through-rank

Key insight: The ML metric and the business metric must be causally linked. Optimizing CTR can hurt revenue if it promotes cheap-but-clickable content.

Always define:

- **Offline metric**: what you optimize during training (log-loss, NDCG)
- **Online metric**: what you measure in A/B tests (CTR, session length)
- **Guardrail metrics**: what must NOT degrade (latency P99, flagged content rate)

Step 3: Data Strategy

Questions to answer:

- Where does training data come from? (logs, human labels, synthetic)
- How is it collected and at what volume?
- What are the label definitions? (positive = click? purchase? 30-second watch?)
- How is it versioned and partitioned?
- What biases exist? (position bias, selection bias, feedback loops)

Data challenges at scale:

- **Positive label scarcity**: 1 purchase per 1000 impressions → class imbalance
- **Label delay**: purchase attribution can lag by 30 days
- **Data leakage**: features computed after the prediction time leak future info
- **Position bias**: items shown in position 1 get clicked more regardless of quality

Step 4: Feature Engineering

Categorize features:

- **User features**: demographics, historical behavior, embeddings
- **Item features**: content features, popularity statistics, embeddings
- **Context features**: device, time, location, session context
- **Cross features**: user-item interaction history, co-occurrence statistics

Online vs. Offline features:

Feature Type	Freshness	Latency	Example
Offline/batch	Hours-days	Low	User's 30-day watch history
Near-real-time	Minutes	Medium	User's last 1-hour behavior
Real-time/online	Seconds	Must be fast	Current session activity

Training-serving skew: The #1 production issue. Occurs when features computed at serving time differ from features used during training. Fix: use the same feature pipeline for both.

Step 5: Model Selection

Choose model complexity based on:

- Latency requirements (simpler = faster)
- Data volume (deep models need >1M examples)
- Feature types (dense = neural; sparse categorical = GBDT or embeddings)
- Explainability needs (regulatory, debugging)

Model progression:

```
Heuristics → Logistic Regression → GBDT → Two-Tower Neural → Full Deep Learning
(baseline)    (fast, interpretable)  (best for tabular)  (embeddings)    (most expressive)
```

Step 6: Training Infrastructure

- **Distributed training**: data parallelism for large data, model parallelism for large models
- **Experiment tracking**: log every run (params, metrics, artifacts)
- **Hyperparameter optimization**: Bayesian search > grid search at scale
- **Retraining frequency**: daily? hourly? triggered by drift?

Step 7: Evaluation

Offline evaluation:

- Hold-out test set (temporally stratified — never shuffle time-series data)
- Cross-validation
- Slice-based analysis (performance per user segment, item category)

Online evaluation:

- A/B test: 90%/10% traffic split, measure business metrics
- Interleaving: merge ranked lists, measure user preferences (faster than A/B)
- Shadow mode: run new model in parallel without affecting users

Step 8: Serving Architecture

Key decisions:

- Batch vs. real-time vs. streaming
- Model hosting: own servers, managed endpoints, edge
- Latency budget: how much time per component?
- Cascading: multiple models at different cost-accuracy tradeoffs

Step 9: Monitoring & Observability

What to monitor:

- **Data health**: input distribution shifts, missing features, schema violations
- **Model health**: prediction distribution, confidence scores
- **Business health**: CTR, revenue, latency P99, error rate
- **Fairness**: performance across demographic slices

Step 10: Iterate

- Close the feedback loop: production predictions → new training data
- Prioritize: data quality improvements often beat model architecture changes
- A/B test every change before full rollout

1.5 Core Concepts

Data Pipelines & Data Versioning

Data pipeline components:

```
Raw Sources → Ingestion → Validation → Transformation → Storage → Feature Computation
(DBs, events)  (Kafka,      (Great        (Spark,          (S3,       (feature store)
               Flink)       Expectations) dbt)            Hive)
```

Data versioning tools:

- **DVC (Data Version Control)**: Git-like versioning for datasets and models. Stores pointers in git, actual data in S3/GCS
- **Delta Lake / Iceberg**: versioned data lakes with ACID transactions, time-travel queries
- **Pachyderm**: data-centric version control with lineage tracking
- **LakeFS**: Git-like branching for data lakes

Why version data?

- Reproduce any past experiment exactly
- Roll back to previous data if quality degrades
- Audit model decisions (regulatory compliance)
- Track data lineage end-to-end

Data validation:

- Schema validation: expected columns, types, ranges
- Statistical validation: mean, variance, null rates within expected bounds
- Distribution shift: compare current batch against historical baseline
- Tools: **Great Expectations**, **TFX Data Validation**, **Deequ**

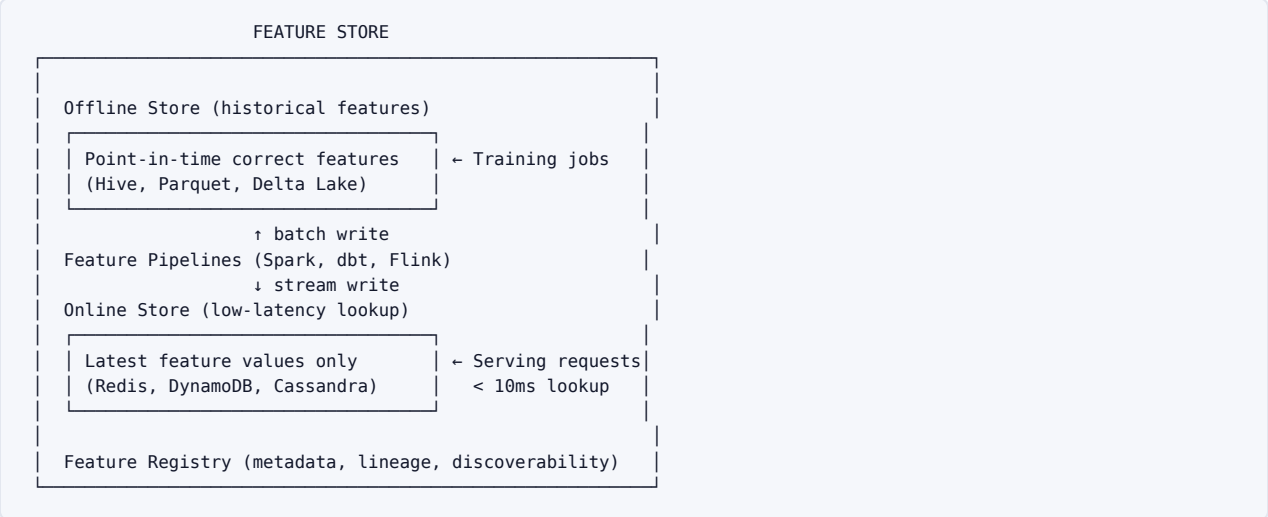
Feature Engineering at Scale & Feature Stores

The feature store problem: Without a feature store, the same feature (e.g., "user's 7-day click rate") is computed independently by:

- The training pipeline (offline batch job)
- The serving system (online real-time computation)

This causes training-serving skew and duplicated engineering work.

Feature store architecture:



Offline store: Historical features for training. Must support **point-in-time correct** joins — when training, we must use only features that would have been available at prediction time (no future leakage).

Online store: Latest feature values for low-latency serving. Redis/DynamoDB for <10ms lookups. Only stores the current value, not history.

Training-serving skew prevention:

- Single feature computation logic used for both offline (batch) and online (stream)
- Log-and-wait: log features used at serving time, join with labels later for training
- Feature validation: statistical tests before training to detect skew

Feature store products:

- **Feast** (open source): original feature store, widely used
- **Tecton**: managed Feast, Uber/Twitter lineage
- **Databricks Feature Store**: integrated with MLflow
- **Vertex AI Feature Store** (GCP): managed, supports streaming
- **SageMaker Feature Store** (AWS): managed, online + offline

Feature freshness vs. cost tradeoff:

Freshness	Update Frequency	Latency	Cost	Use Case
Batch	Daily/hourly	Low	Low	Long-term user preferences
Near-real-time	Minutes	Medium	Medium	Recent session behavior
Real-time	Seconds	< 10ms required	High	Current context features

Experiment Tracking & Model Registry

Experiment tracking captures:

- **Parameters**: hyperparameters, model architecture config
- **Metrics**: training/validation loss, AUC, NDCG at each epoch
- **Artifacts**: model weights, feature importance plots, confusion matrices
- **Environment**: Docker image, library versions, hardware specs
- **Code**: git commit hash

Tools:

- **MLflow**: open-source, widely adopted; tracking + registry + projects + models
- **Weights & Biases (W&B)**: rich visualization, popular in deep learning
- **Neptune.ai**: collaborative experiment tracking
- **Vertex AI Experiments** / **SageMaker Experiments**: managed cloud options

MLflow workflow:

CODE

```
with mlflow.start_run():
    mlflow.log_param("learning_rate", 0.001)
    mlflow.log_param("n_estimators", 100)

    # Train model...

    mlflow.log_metric("val_auc", 0.87)
    mlflow.sklearn.log_model(model, "model")

# Register best model
mlflow.register_model("runs:/abc123/model", "CTR_Predictor")
```

Model Registry manages the model lifecycle:

```
┌──────────────────────────────────────────────────────┐
│              MODEL REGISTRY                          │
│                                                      │
│  Stages: Staging → Validation → Production → Archived │
│                                                      │
│  For each model version:                             │
│  - Model artifact (weights, preprocessing pipeline)  │
│  - Training metadata (data version, parameters)      │
│  - Evaluation metrics (offline + online)             │
│  - Deployment history                                │
│  - Approval workflow (human sign-off for production) │
└──────────────────────────────────────────────────────┘
```

Key registry capabilities:

- **Lineage**: which data + code produced this model?
- **Versioning**: compare model versions side by side
- **Governance**: approval gates before production promotion
- **Rollback**: one-click revert to previous version

Distributed Training

Why distributed?

- Model too large for one GPU (GPT-3: 175B params, 350GB in fp32)
- Data too large to train on one machine in reasonable time
- Need faster experimentation turnaround

Three parallelism paradigms:

Data Parallelism

```
          Master (gradient aggregation)
               /         |         \
         Worker 1    Worker 2    Worker 3
         [Shard 1]  [Shard 2]  [Shard 3]
         full model  full model  full model
```

- Each worker holds a **full copy of the model**
- Data is split across workers (each sees different mini-batches)
- Workers compute gradients independently
- Gradients aggregated (sum/average) and model weights updated
- **Works for**: models that fit in one GPU's memory

All-Reduce vs. Parameter Server:

Architecture	All-Reduce	Parameter Server
Gradient aggregation	Ring all-reduce (each worker sends/receives)	Central PS node aggregates
Communication pattern	Peer-to-peer	Hub-and-spoke
Bottleneck	Network bandwidth	PS becomes bottleneck
Fault tolerance	Worker failure affects ring	PS can be replicated
Best for	Homogeneous GPU clusters	Heterogeneous, sparse models
Example	Horovod, PyTorch DDP	TensorFlow PS, Uber Petastorm

Ring All-Reduce (NCCL):

```
W1 → W2 → W3 → W4
↑               ↓
W4 ← W3 ← W2 ← W1
```

Each worker sends a chunk to the next, receives a chunk from the previous. After N-1 steps, each worker has the full gradient sum. Bandwidth-optimal: each worker sends/receives exactly one full gradient tensor total.

Model Parallelism

```
Worker 1        Worker 2        Worker 3
[Layers 1-4]   [Layers 5-8]   [Layers 9-12]
    ↓ activations ↓ activations ↓
```

- Model is **split across workers** by layer or module
- Each worker holds only a portion of the model
- Forward pass: activations flow from worker to worker
- **Works for**: models too large for a single GPU (LLMs)
- **Challenge**: pipeline bubbles — workers idle while waiting for previous stage

Pipeline Parallelism (GPipe, PipeDream):

```
Micro-batch 1: [W1 compute] → [W2 compute] → [W3 compute]
Micro-batch 2:               [W1 compute] → [W2 compute] → [W3 compute]
Micro-batch 3:                              [W1 compute] → [W2 compute] → ...
```

Split model into stages + split mini-batch into micro-batches to fill the pipeline.

Tensor Parallelism

- Split individual **weight matrices** across GPUs
- Each GPU computes part of a matrix multiply in parallel
- Used by Megatron-LM for extremely large transformers
- Requires high-bandwidth GPU interconnects (NVLink)

Combined: 3D Parallelism (Megatron + DeepSpeed)

```
Data Parallel × Pipeline Parallel × Tensor Parallel
= can train 1 trillion parameter models
```

DeepSpeed ZeRO (Zero Redundancy Optimizer): Partitions optimizer states, gradients, and parameters across data-parallel workers, dramatically reducing per-GPU memory.

ZeRO Stage	What's Partitioned	Memory Reduction
Stage 1	Optimizer states	4x
Stage 2	Optimizer states + gradients	8x
Stage 3	Optimizer states + gradients + parameters	64x

Model Serving

Three serving paradigms:

Paradigm	Latency	Throughput	Use Case	Example
Batch inference	Hours	Very high	Precomputed embeddings, offline scoring	Weekly product recs
Online/real-time	< 100ms	Medium	Interactive queries, search ranking	Search results
Streaming	Seconds	High	Continuous event processing	Fraud detection on transactions

Batch vs. Online serving deep dive:

Aspect	Batch	Online
Trigger	Schedule (cron)	User request
Latency SLA	None (hours)	Strict (< 100ms)
Compute cost	Cheaper (spot instances)	Higher (always-on)
Feature freshness	Stale (hours-days)	Fresh (milliseconds)
Model complexity	Any	Limited by latency budget
Infrastructure	Spark, Beam	FastAPI, TorchServe, TF Serving
Failure handling	Retry easily	Circuit breakers, fallbacks

Serving stack components:

```
Client Request
      ↓
Load Balancer
      ↓
API Gateway (auth, rate limiting)
      ↓
Feature Service → Online Feature Store (Redis)
      ↓
Model Server (TF Serving / TorchServe / Triton)
      ↓           ↑
   GPU/CPU     Model Registry (pulls model artifacts)
      ↓
Post-processing (business rules, safety filters)
      ↓
Response
```

GPU serving optimizations:

- **Batching**: group multiple requests into one GPU forward pass (increases throughput, adds latency)
- **Model quantization**: FP32 → INT8 (4x speedup, ~1% accuracy drop)
- **TensorRT**: NVIDIA's inference optimizer — fuses layers, optimizes compute graphs
- **ONNX**: model exchange format — train in PyTorch, serve with any runtime
- **Dynamic batching**: TorchServe, Triton aggregate requests within a time window
- **KV-cache** (LLMs): cache attention key-value pairs for auto-regressive generation

Latency vs. throughput tradeoff:

- Small batch size → low latency, low throughput
- Large batch size → high throughput, high latency
- **Batching timeout**: ”wait up to 5ms for more requests, then process whatever we have”

Model serving frameworks:

Framework	Best For	Key Feature
TensorFlow Serving	TF models, production-grade	gRPC, REST, versioning
TorchServe	PyTorch models	Custom handlers, dynamic batching
NVIDIA Triton	Any framework, GPU-first	Ensemble pipelines, GPU optimization
BentoML	ML-framework agnostic	Python-native, easy packaging
Ray Serve	Python-native, distributed	Actor-based, online learning
vLLM	LLM inference	PagedAttention, continuous batching

Model-as-a-Service patterns:

- **Shadow deployment**: new model receives traffic copies but responses are discarded. Validates model works without risk
- **Canary deployment**: send 1-5% of traffic to new model, gradually increase
- **Blue-green**: maintain two identical environments, switch traffic instantly
- **Multi-armed bandit**: dynamically allocate traffic to better-performing variants

A/B Testing & Online Experimentation

Why A/B test? Offline metrics (AUC, NDCG) don't always translate to business metrics (revenue, engagement). Online experiments close this gap.

A/B test anatomy:

```
Traffic → [Experiment Assignment] → Control (A) or Treatment (B)
                                        ↓                ↓
                                  Log responses   Log responses
```

```
                ↓
Statistical analysis
(t-test, Mann-Whitney, bootstrap)
                ↓
Ship if: p < 0.05 AND effect size meaningful
```

Statistical considerations:

- **Sample size**: determined by minimum detectable effect (MDE), alpha (Type I error), beta (Type II error / power)
- **Multiple testing problem**: testing 20 metrics at p=0.05 expects 1 false positive. Use Bonferroni correction or FDR control
- **Network effects (interference)**: user A's experience affects user B's (social networks). Solution: cluster randomization
- **Novelty effect**: users engage more with anything new. Run experiments for 2+ weeks

Experiment designs for ML:

Method	Description	Speed	Use Case
A/B test	Split users into control/treatment	Slow (weeks)	Standard
Interleaving	Merge ranked lists from A and B	100x faster	Ranking systems
Multi-armed bandit	Dynamic traffic allocation	Adaptive	When exploration-exploitation matters
Switchback	Alternate between A/B by time period	Fast	Marketplace, supply-side effects

Interleaving (for recommendation/search):

- Merge recommendation lists from model A and model B, randomly shuffling ties
- Measure which model's items users click more
- Much more sensitive than A/B (each user is their own control)
- Used by Netflix, Spotify, LinkedIn

Guardrail metrics — metrics that must not degrade even if primary metrics improve:

- Page load time
- Error rate
- Revenue per user
- Content flagging rate

Monitoring & Observability

Four types of drift:

Drift Type	What Changes	Detection Method	Example
Data drift	Input feature distribution P(X)	KL divergence, PSI, Kolmogorov-Smirnov	User demographics shift after product expansion
Concept drift	Relationship P(Y|X)	Monitor prediction accuracy on new labels	COVID changed search intent for ”mask”
Label drift	Label distribution P(Y)	Monitor label statistics	Fraud patterns shift seasonally
Prediction drift	Model output distribution P(Ŷ)	Monitor prediction distribution	Sudden spike in high-confidence predictions

Population Stability Index (PSI):

```
PSI = Σ (Actual% - Expected%) × ln(Actual% / Expected%)

PSI < 0.1: No significant change
PSI 0.1-0.2: Moderate change, monitor
PSI > 0.2: Major shift, investigate
```

Monitoring stack:

```
Production Traffic
      ↓
Feature Logging → Data Store (Kafka → S3)
      ↓
Monitoring Pipeline (Spark Streaming / Flink)
      │
      ├── Data Quality Checks (nulls, schema, range violations)
      ├── Statistical Drift Detection (PSI, KS-test, chi-squared)
      ├── Prediction Distribution Monitoring
      └── Business Metric Dashboards (CTR, revenue, latency)
           ↓
      Alert System (PagerDuty, Slack)
           ↓
      [Automatic retraining trigger] OR [Human investigation]
```

Model decay causes:

- **Covariate shift**: P(X) changes, P(Y|X) unchanged (feature distribution changes)
- **Concept drift**: P(Y|X) changes (the underlying relationship changes)
- **Feedback loops**: model predictions influence future training data
- **Upstream data changes**: schema changes, pipeline failures, new null patterns

Observability pillars for ML:

- **Logs**: every prediction logged with features and context
- **Metrics**: aggregated statistics (mean prediction, P95 latency, error rate)
- **Traces**: end-to-end request tracing from API call to model response
- **Alerts**: threshold-based and anomaly-detection-based

Tools:

- **Evidently AI**: open-source data/model monitoring, drift reports
- **Arize AI**: commercial ML observability
- **WhyLabs**: statistical profiling and monitoring
- **Fiddler AI**: explainability + monitoring
- **Prometheus + Grafana**: metrics and dashboards (infrastructure layer)

CI/CD for ML (Continuous Training)

Standard CI/CD vs. ML CI/CD:

Stage	Software CI/CD	ML CI/CD
Trigger	Code push	Code push OR new data OR drift alert
Build	Compile, lint	Data validation, feature pipeline
Test	Unit tests, integration tests	Model evaluation, slice analysis, bias checks
Artifact	Docker image	Model weights + preprocessing pipeline
Deploy	Container rollout	Canary model rollout
Monitor	Uptime, error rate	Prediction quality, data drift

Continuous Training (CT):

- Automatically retrain when triggered by: schedule, data threshold, or drift detection
- Requires: automated data validation, automated evaluation gates, automated deployment
- **Full CT pipeline**:

```
Trigger → Data Pipeline → Validation → Training → Evaluation
                                                      ↓
          [Fail: alert humans]  ←  [Pass threshold?]  ←
                                          ↓ Yes
                            Model Registry (staging)
                                          ↓
                            Automated integration tests
                                          ↓
                            Shadow deployment
                                          ↓
                            Canary deployment (5%)
                                          ↓
                            Full rollout (if metrics OK)
```

Reproducibility requirements:

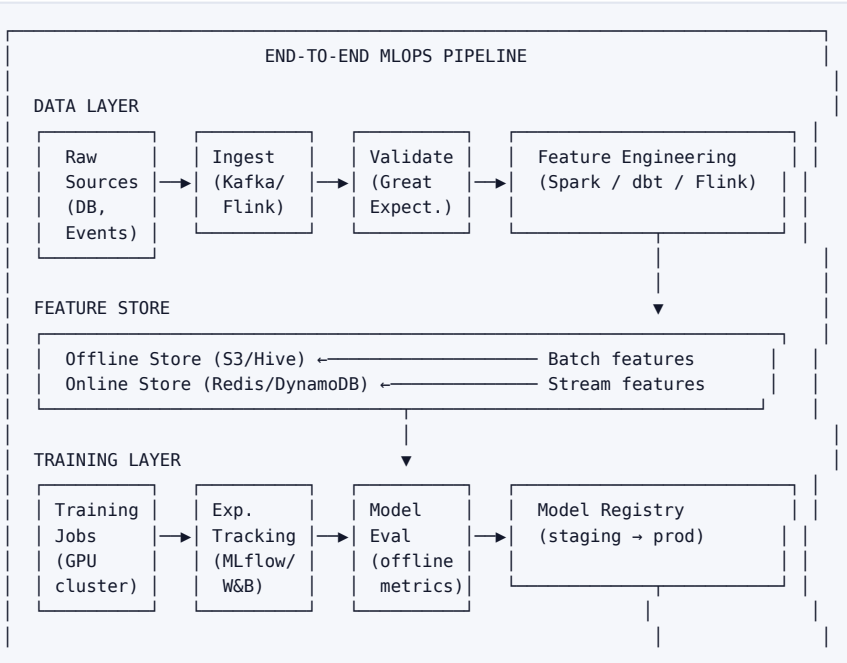
- **Code**: git commit hash
- **Data**: data version (DVC tag, Delta Lake version)
- **Environment**: Docker image with pinned dependencies
- **Hyperparameters**: logged in experiment tracker
- **Random seeds**: fixed for reproducibility

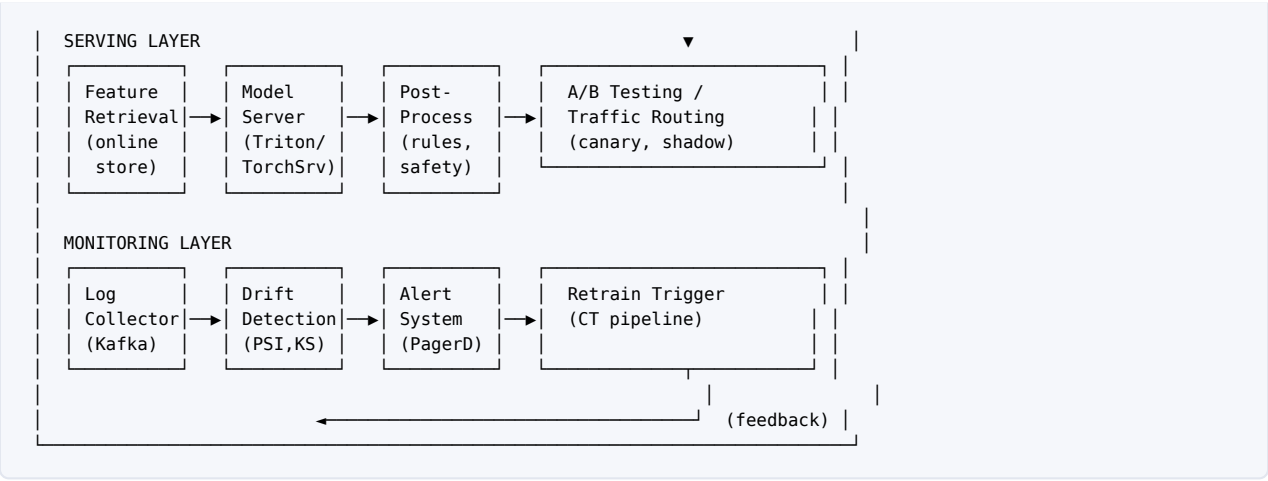
ML pipeline orchestration tools:

- **Apache Airflow**: general DAG orchestration, widely used
- **Kubeflow Pipelines**: Kubernetes-native ML pipelines
- **MLflow Projects**: package code with conda/docker environments
- **Metaflow** (Netflix): Python-native, scales from laptop to AWS
- **Prefect / Dagster**: modern Python-native orchestration
- **Vertex AI Pipelines**: GCP managed, Kubeflow-compatible

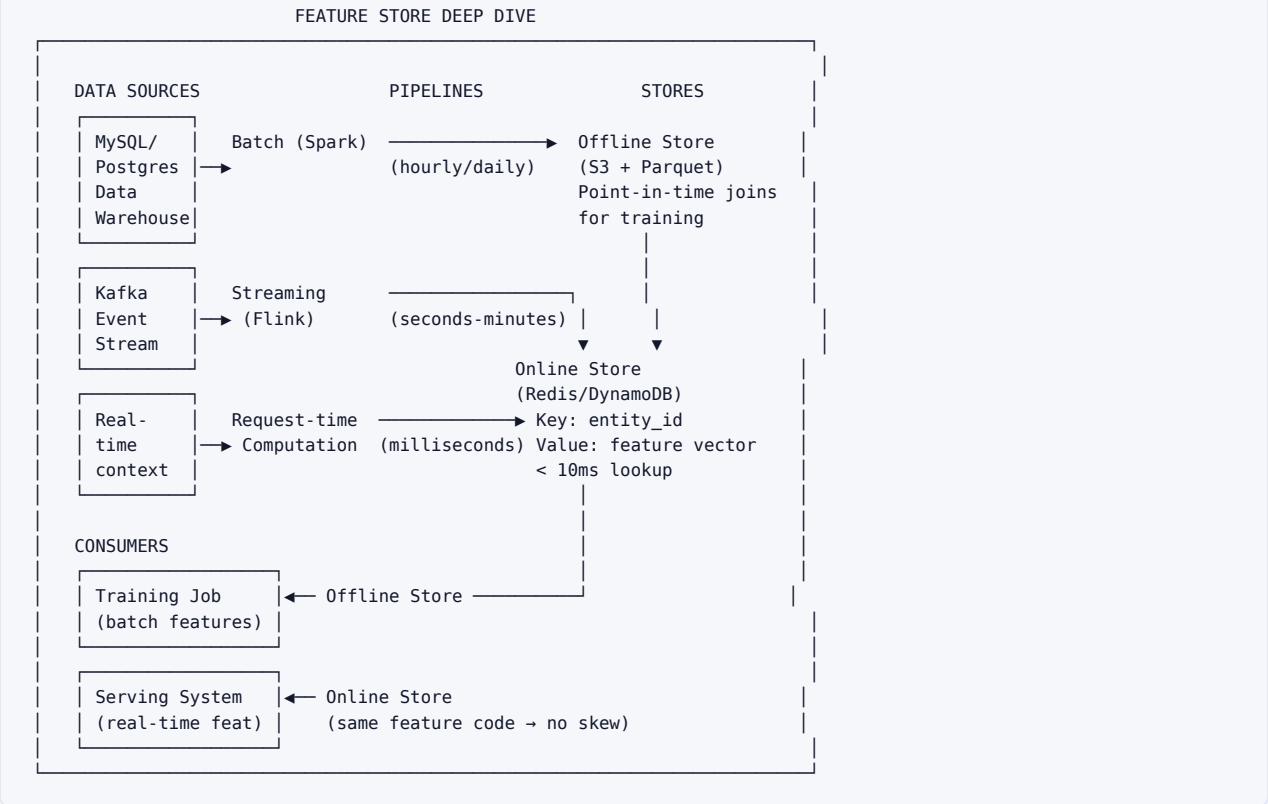
1.6 Architecture / Diagrams

End-to-End MLOps Pipeline

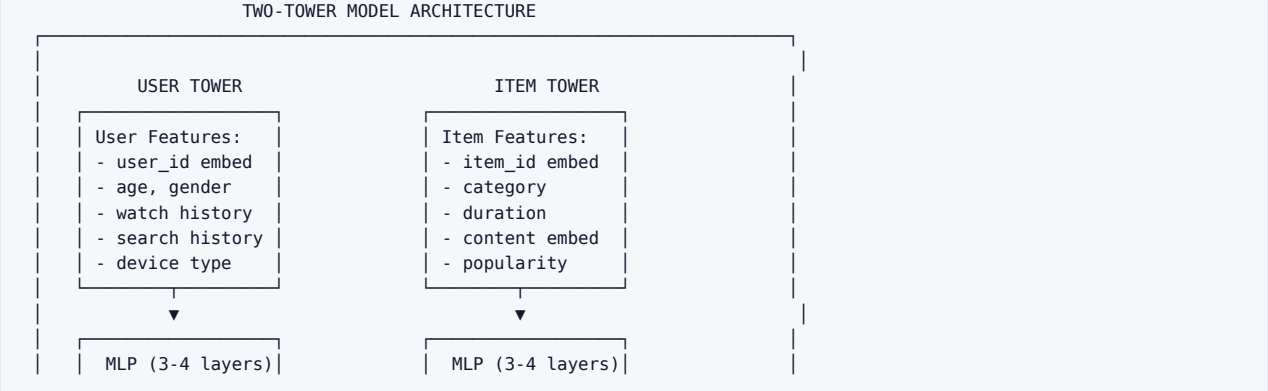


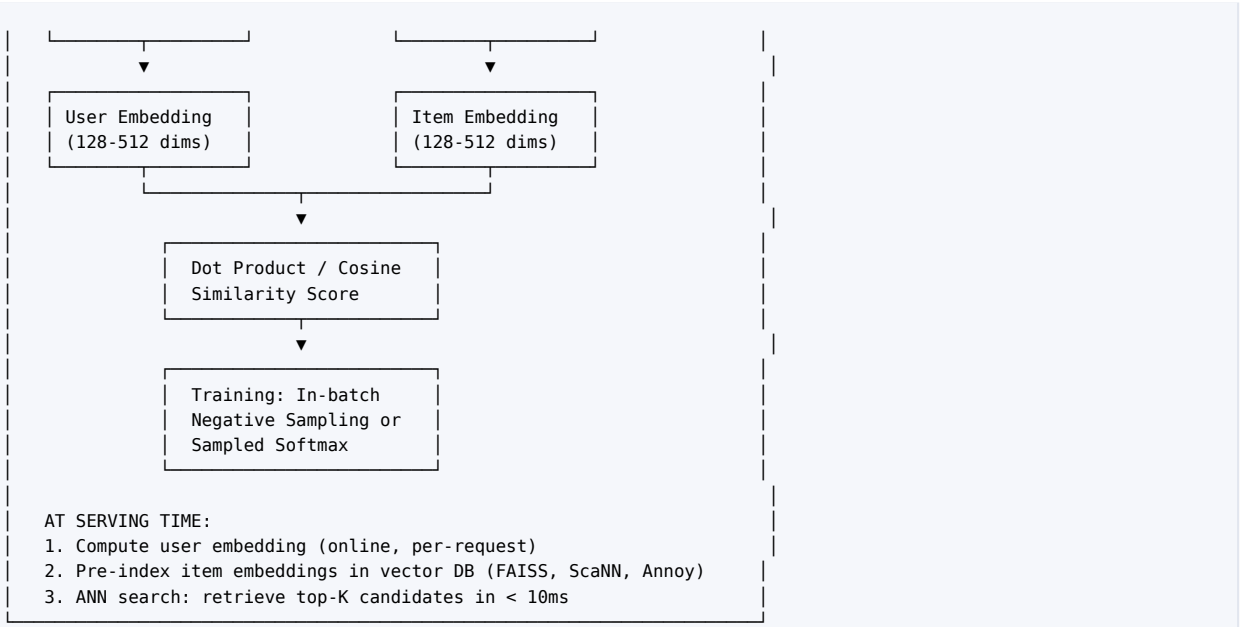


Feature Store Architecture (Online/Offline)

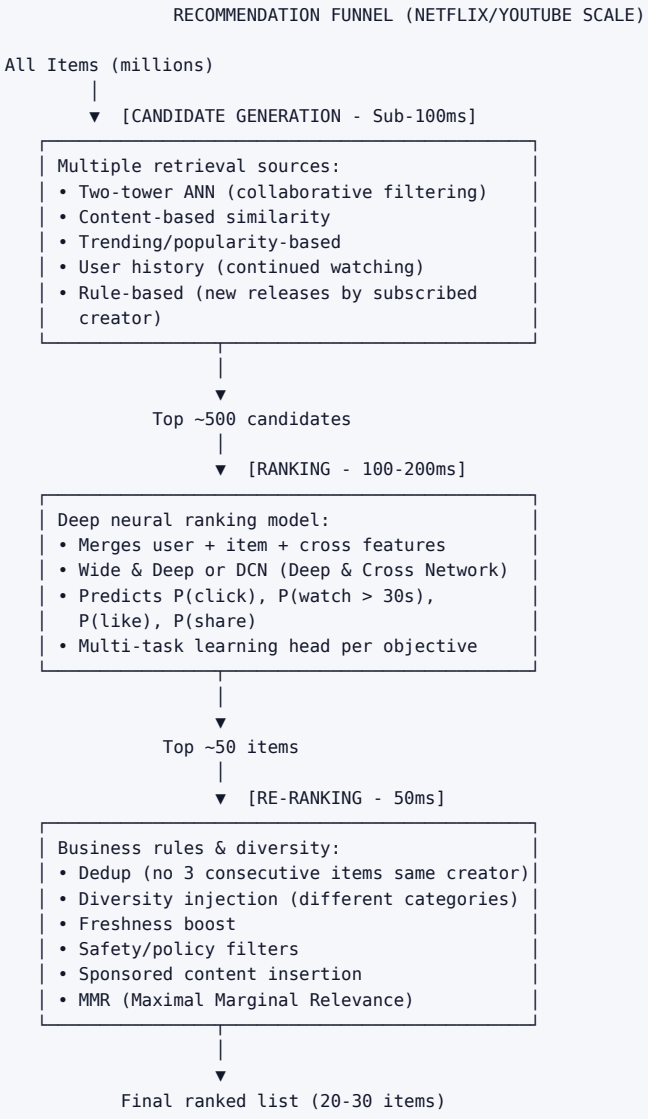


Two-Tower Recommendation Architecture





Candidate Generation → Ranking Funnel



```
                  |
                  ▼
              User sees feed
```

Data vs. Model Parallelism

```
DATA PARALLELISM
────────────────

GPU 1: [Model copy] ← [Shard 1 of data]  →  gradients ─┐
GPU 2: [Model copy] ← [Shard 2 of data]  →  gradients  ├─ All-Reduce → Updated model
GPU 3: [Model copy] ← [Shard 3 of data]  →  gradients ─┘
GPU 4: [Model copy] ← [Shard 4 of data]  →  gradients ─┘

Constraint: Entire model must fit on one GPU


MODEL PARALLELISM (Pipeline)
────────────────────────────

Micro-batch flows through pipeline stages:

GPU 1: [Layers 1-3]  →  activations
                              ↓
GPU 2:             [Layers 4-6]  →  activations
                                          ↓
GPU 3:                       [Layers 7-9]  →  activations
                                                    ↓
GPU 4:                                [Layers 10-12] → loss

Allows training models that don't fit on one GPU


TENSOR PARALLELISM
──────────────────

Matrix multiply W × X split across GPUs:

GPU 1: W[rows 0:N/2, :] × X = partial result 1 ─┐
GPU 2: W[rows N/2:N, :] × X = partial result 2  ─┴─ AllGather → full result
```

Drift Monitoring Loop

```
DRIFT MONITORING & RESPONSE LOOP

Production ──▶ Feature Logging ──▶ Data Store (S3)
   │                                      │
   │                                      ▼
   │                           Statistical Tests
   │                           ┌──────────────────────┐
   │                           │ PSI per feature      │
   │                           │ KS-test distributions│
   │                           │ Chi-squared (categ.) │
   │                           │ Prediction dist.     │
   │                           └──────────┬───────────┘
   │                                      │
   │                                 Drift detected?
   │                                /              \
   │                              NO                YES
   │                              │                 │
   │                     Continue                Severity?
   │                    monitoring              /        \
   │                                          LOW         HIGH
   │                                           │           │
   │                                 Log & notify   Auto-retrain
   │                                 team (Slack)   pipeline OR
   │                                                page on-call
   │                                                    │
   └────────────────────────────────────────────────────┘
          (retrained model pushed back to prod)
```

1.7 Worked Examples

Design a Recommendation System (e.g., YouTube / Netflix)

Clarify requirements:

- 1B+ users, 100M+ videos, 5M QPS at peak
- Latency: < 200ms end-to-end
- Goal: maximize long-term user satisfaction (not just next click)
- Cold start: handle new users, new videos

ML Objective:

- Multi-task: predict P(click), P(watch > 50%), P(like), P(share)
- Weighted combination as ranking score
- Metric: user satisfaction proxy (watch time, explicit rating), not just CTR

Data:

- Implicit feedback: watch events, clicks, skips, shares (billions/day)
- Explicit feedback: ratings, thumbs up/down (sparse)
- Item metadata: transcripts, thumbnails, category, creator
- Labels: positive = watched > 50% of video, negative = skipped < 30s

Features:

Category	Features
User	user_id embedding, age, country, subscription tier
User behavior	watched_video_ids (last 50), search_queries (last 10), avg_watch_time
Item	video_id embedding, category, duration, language, upload_date
Item popularity	view_count_24h, like_rate, comment_rate
Context	time_of_day, device_type, session_number
Cross	user's historical engagement with this creator/category

Model:

1. **Candidate Generation (Two-Tower)**:
   - User tower: embed user_id + process user features → 256-dim user embedding
   - Item tower: embed video_id + process item features → 256-dim item embedding
   - Training: sampled softmax over all items in batch
   - Serving: pre-compute item embeddings, index in FAISS/ScaNN for ANN retrieval
   - Retrieve top 500 candidates < 10ms
2. **Ranking (Wide & Deep or DCN-v2)**:
   - Inputs: 500 candidates × user features × cross features
   - Multi-task output heads: P(click), P(watch50%), P(like)
   - Final score: weighted sum of task predictions + calibration
   - Latency: < 100ms for 500 candidates
3. **Re-ranking**:
   - Diversity: MMR to avoid consecutive same-creator videos
   - Freshness: boost videos < 24h old
   - Safety: filter policy-violating content
   - Promoted content: insert ads with guaranteed placements

Training:

- Training data: last 7 days of user interaction logs
- Retrain: daily (with freshness signals updated hourly via streaming)
- Distributed: data parallel training on 64 A100 GPUs

Evaluation:

- Offline: AUC for each prediction task, NDCG@10, calibration (expected vs. actual CTR)
- Online: A/B test measuring total watch time, session length, 7-day retention
- Guardrails: creator fairness (no single creator dominates), content diversity

Monitoring:

- Feature drift: PSI on user and item feature distributions daily
- Prediction drift: monitor mean/variance of CTR predictions
- Retraining trigger: if offline AUC drops > 0.5% OR drift PSI > 0.2 on key features

Design CTR (Click-Through Rate) Prediction for Ads

Context: This is the core revenue model at Google, Meta, Amazon ads.

Clarify requirements:

- Scale: 10M ads, 3B users, 5M QPS
- Latency: < 50ms (ads integrated into search/feed, strict latency SLA)
- Objective: accurately predict probability a user will click an ad
- Revenue: auction price = bid × predicted CTR (Vickrey auction)

ML Objective:

- Binary classification: P(click | user, ad, context)
- Key metric: **calibration** (critical — mispredicted CTR distorts auction prices)
- Also track AUC-ROC (discrimination ability)

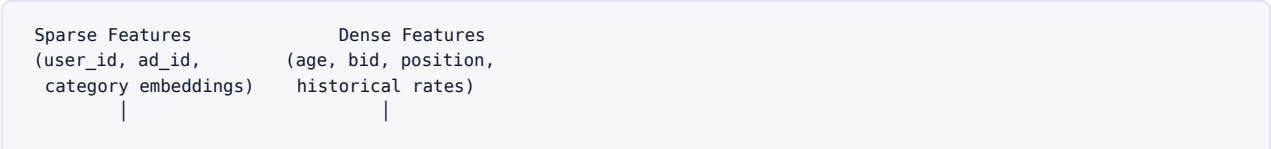
Data:

- Billions of (impression, user, ad) tuples per day
- Label: click (positive) / no-click (negative)
- Extreme class imbalance: 1 click per 200-1000 impressions
- Label strategy: downsample negatives to 1:50 or use weighted loss

Features:

Category	Features	Freshness
User	age, gender, interest categories, past ad click history	Daily batch
User real-time	Last 5 ads clicked this session, last search query	Real-time
Ad	ad_id, advertiser, category, creative type, bid price	Ad metadata
Query/context	search query embeddings, page topic, device, time	Per-request
Cross features	user-ad category match, query-ad relevance	Per-request
Historical	user CTR for this ad, CTR for advertiser, position stats	Daily batch

Model:
DLRM (Deep Learning Recommendation Model) or **Wide & Deep**:



Calibration is critical: if model predicts CTR = 5% but actual is 2.5%, ads are underpriced by 2x.

- Platt scaling, isotonic regression post-hoc calibration
- Regular recalibration as audience/ad inventory changes

Serving:

- Retrieval: ad candidates selected by bid × predicted CTR (simplified GSP auction)
- Latency breakdown: feature retrieval (10ms) + model inference (20ms) + auction logic (10ms) = 40ms
- Hardware: dedicated GPU inference cluster; INT8 quantized models
- Caching: precompute user embeddings hourly, cache in Redis

Training:

- Frequency: retrain daily with last 24h of fresh data; fine-tune every hour on last 1h
- Distributed: data parallel, 32-128 GPUs
- Exploration: ε-greedy or Thompson sampling for ad exploration

Monitoring:

- **Online calibration**: compare predicted CTR vs. actual CTR in rolling 1-hour windows
- **Revenue per query**: primary business metric
- **Prediction drift**: sudden shift in CTR predictions can indicate data pipeline issue
- **Latency P99**: ads must not delay page load

Design News Feed Ranking (Facebook / Twitter / LinkedIn)

Context: Orders millions of posts per user per day into a feed that maximizes engagement and wellbeing.

Clarify:

- 3B users, 100B posts/day created, user sees ~100 posts per session
- Latency: < 500ms (user opens app, sees feed)
- Multi-objective: engagement AND user wellbeing AND advertiser revenue AND creator success

ML Objective:

- Multi-task prediction per candidate post: P(like), P(comment), P(share), P(hide/report), P(reading time)
- Combined score: weighted sum with business weight tuning
- Negative signals (hide, report) heavily penalized

Data:

- Social graph: who follows whom, friendship strength
- Engagement logs: all interactions per post per user (100B events/day)
- Content: text, images, video, links with metadata
- Temporal: recency matters — 1-hour old post vs. 1-day old post

Architecture:

```
FEED RANKING PIPELINE

Social Graph + All Candidate Posts (last 7 days from network)
                    |
                    ▼ [HEURISTIC FILTERING - <10ms]
          Reduce to top 2000 posts
          (recency filter, basic relevance)
                    |
                    ▼ [LIGHT RANKER - <50ms]
          Simple model (logistic regression or small DNN)
          Reduces to top 500 candidates
                    |
                    ▼ [HEAVY RANKER - <300ms]
          Large multi-task neural network
          Predicts: P(like), P(comment), P(share),
                    P(hide), P(30s read), P(click link)
          Features: user + post + social context + cross
                    |
                    ▼ [CONTEXTUAL ADJUSTMENT - <50ms]
          Diversity rules (max 3 consecutive from same user)
          Ads insertion (every 5th item)
          Promoted content
          Safety filters
                    |
                    ▼
          Final feed (100 posts, paginated 20/load)
```

Key Features:

Type	Example Features
Post content	text length, image/video, sentiment, entity mentions
Post freshness	time since posting (exponential decay weighting)
Creator	creator_id embed, follower count, past post engagement rate
User-creator relationship	interaction frequency last 30 days, recency of last interaction
Social signals	how many of user's friends engaged with this post
User session	time of day, session length, last 10 posts interacted with

Multi-objective balancing:

- Naively maximizing engagement drives clickbait and outrage
- Meta's approach: add ”meaningful social interactions” signal (comments > reactions > likes)
- LinkedIn: weight toward professional content, career-relevant posts
- Explicit tuning: product managers adjust weights between objectives

Training:

- Labels: delayed engagement (collect for 24h post-impression, not just immediate)
- Negative labels: explicitly marked as ”hide post” or ”see fewer like this”
- Training frequency: daily full retrain + hourly fine-tuning
- Evaluation: offline NDCG + online A/B test of session length, DAU, weekly active engagement

Monitoring:

- Content diversity index (are users seeing a filter bubble?)
- Misinformation spread rate (flagged content in top positions)
- Creator fairness (small creators vs. large creators reach)
- Engagement health vs. wellbeing indicators

Design Search/Ranking System (Google Search / Bing / Amazon Product Search)

Key differences from recommendations:

- User provides explicit query — query understanding is critical
- Relevance is primary, personalization is secondary
- Both precision (first result correct) and recall (don't miss relevant docs) matter

Pipeline:

```
Query → Query Understanding → Retrieval → Ranking → Results
        (spell correct,        (BM25,     (neural    (snippets,
        intent, NER)           dense      ranker)     ads)
                               retrieval)
```

Query understanding:

- Spell correction (Levenshtein, BERT-based)
- Query classification: informational, navigational, transactional
- Named entity recognition: ”Nike shoes” → brand + product type
- Query expansion: synonyms, related terms

Retrieval (first-stage):

- **BM25**: lexical matching, fast, interpretable
- **Dense retrieval**: bi-encoder (query and document into same embedding space), ANN search
- **Hybrid**: combine BM25 + dense scores

Neural reranking (second-stage):

- **Cross-encoder**: concatenate query + document, run through BERT → relevance score
- Much more accurate than bi-encoder (sees full interaction)
- Too slow for full corpus → apply only to top-100 from first stage

Learning to Rank:

- **Pointwise**: predict relevance score for each document independently
- **Pairwise**: learn which of doc A / doc B is more relevant (RankNet, LambdaRank)
- **Listwise**: optimize a list-level metric directly (LambdaMART, AdaRank)

1.8 Real-World Examples

Netflix:

- Candidate generation: two-tower model retrieves ~1000 candidates per user
- Ranking: uses estimated ”take rate” (will user choose to watch the item if they see the row)
- Evaluation: primarily causal inference — A/B tests measuring streaming hours
- Feature store: Hollow (internal) — manages billions of features across user and item spaces
- Monitoring: model staleness tracked daily; immediate alert if prediction distributions shift > 2σ

Google:

- Ad CTR model: billions of training examples daily; retrained multiple times per day
- Feature engineering: Cross features (user $\times$ ad) via feature crosses in Wide & Deep
- Calibration: click models include position bias correction
- Serving: custom ASIC hardware (TPUs) for inference at scale

Uber:

- Michelangelo platform: end-to-end ML platform — feature store, training, serving, monitoring
- Feature store: Hive (offline) + Cassandra (online) with shared feature computation

- Use case: ETA prediction, surge pricing, matching driver to rider

Meta:

- News feed: multi-task model with 12+ prediction heads
- Feature store: every user feature updated in real-time as users interact
- Interleaving experiments: run thousands of simultaneous experiments
- DLRM (open sourced): their production recommendation model architecture

LinkedIn:

- Feed ranking: uses ”expected dwell time” as primary signal, not clicks
- Feature: Economic Graph embeddings (job title, company, skill entity embeddings)
- Experiment: Every feature, model change goes through strict A/B framework

DoorDash:

- ETA model: predicts restaurant prep time + courier travel time
- Key MLOps challenge: models degrade on holidays/events — specialized retraining triggers
- Training-serving skew: courier GPS features differ between training logs and real-time serving

1.9 Real-Life Analogies --- An Airport Operations Center Running Flights at Scale

Every piece of MLOps maps to the controlled chaos of keeping thousands of flights departing on time from a major hub like Chicago O'Hare.

MLOps Component	Airport Analog	Why The Parallel Works
Data pipeline	Baggage conveyor system	Raw bags (events) ingested, sorted, routed to the right destination (data store) — constantly moving, with error handling for mislabeled bags (schema violations)
Feature store (offline)	Pre-prepped airline meal depot	Meals prepared hours ahead in batches, stored by route type; training uses these historical meal logs to know what passengers on Route X typically ate
Feature store (online)	Fuel depot at the gate	Real-time fuel requested per specific flight, delivered in seconds before departure; must be fresh and correct or the flight is grounded
Model registry	Certified aircraft archive	Every aircraft model has a certification record — airworthiness tests passed, maintenance history, approved for which routes. You only fly certified aircraft
Training	Pilot training in flight simulators	Pilots train on simulated scenarios (historical data); new conditions require recertification (retraining)
Serving (real-time)	Live flights departing on schedule	Gate assignments, departure sequences — decisions made in seconds, must be correct, must scale to 1000 daily flights
Batch serving	Pre-printed passenger manifests	Generated hours before the flight for all passengers; fine to be slightly stale
A/B testing	Testing two runway configurations	North runway vs. south runway: measure average taxi time, fuel burn, delays. Switch all traffic to winner
Monitoring/drift	Control tower watching weather change	Tower continuously monitors conditions; if wind pattern shifts (drift), immediately reroutes aircraft or triggers holds
Model decay	Flight routes becoming outdated	Routes planned for old demand patterns; if a city grows (distribution shift), the old route plan serves it poorly
Retraining	Recertifying crews after new regulations	FAA changes rules (concept drift); all crews must be retrained and recertified before flying under new conditions
CI/CD pipeline	Pre-flight safety checklist**	Before any flight departs (model deploys), the same rigorous checklist is run every time — no shortcuts

MLOps Component	Airport Analog	Why The Parallel Works
Latency (P99)	On-time departure rate	P99 latency = the flight that takes longest even on a bad day; 99th percentile delays cascade through the hub
Data versioning	Black box flight recorder	Every flight records exactly what happened; if a crash occurs (model failure), you replay the recording to find the root cause
Shadow deployment	Test flight with no passengers	New aircraft configuration runs a full route but carries no revenue passengers — validates before trusting with real load
Canary release	Single gate uses new boarding process	Try new boarding procedure on 2 flights/day; if it works, roll out to all 500 gates

The throughput goal: every flight on time, safe, efficient. The ML goal: every prediction timely, accurate, at scale.

1.10 Memory Tricks / Mnemonics

The Framework: "CODFMTESMI" Say: "**C**hef **O**rders **D**ishes **F**resh **M**ornings — **T**aste, **E**at, **S**erve, **M**onitor, **I**terate"

- **C**larify, **O**bjective, **D**ata, **F**eatures, **M**odel, **T**rain, **E**val, **S**erve, **M**onitor, **I**terate

Drift Types: "DCPL"

- **D**ata drift (X changes)
- **C**oncept drift (Y|X changes)
- **P**rediction drift ($\hat{Y}$ changes)
- **L**abel drift (Y distribution changes)

Parallelism: "DMP = Different Model Parts"

- **D**ata parallelism: same model, different data
- **M**odel parallelism: different model layers, same data
- **P**ipeline parallelism: model stages pipelined like factory assembly line

Feature Store: "ROPE"

- **R**eal-time (online store, Redis)
- **O**ffline (batch features, S3/Hive)
- **P**oint-in-time correct (no future leakage in training)
- **E**liminate skew (same code for train and serve)

Serving modes: "BOS" = Batch, Online, Streaming

- **B**atch: precomputed (hours old is fine)
- **O**nline: request-triggered (< 100ms)
- **S**treaming: event-driven (seconds latency)

Recommendation funnel: "CRRR"

- **C**andidate generation (millions $\rightarrow$ hundreds)
- **R**anking (hundreds $\rightarrow$ top 50)
- **R**e-ranking (business rules, diversity)
- **R**esults served to user

A/B test validity: "SMPN"

- **S**ample size (pre-calculated, don't peek early)
- **M**ultiple testing correction (Bonferroni)

- **P**ower (80%+ power)
- **N**ovelty effect (run 2+ weeks)

PSI Thresholds: ”10-20-Go”

- < 0.10 = stable
- 0.10-0.20 = monitor
-

> 0.20 = action required

1.11 Common Interview Questions

Q1: "Design a recommendation system for Netflix."

Model answer framework:

1. Clarify: 200M users, 20K titles, personalized home page, < 200ms, maximize watch time
2. ML objective: predict P(user watches title > 50%), optimize ranked list for total watch time
3. Data: watch history (positive), skips < 30s (negative), explicit ratings
4. Features: user watch history embeddings, title content embeddings, temporal features
5. Model: Two-tower candidate retrieval → Deep ranking model → Re-ranking
6. Training: Daily retrain, data parallel on GPU cluster
7. Evaluation: Offline NDCG + Online A/B (watch hours, retention)
8. Monitoring: PSI on feature distributions, weekly model accuracy evaluation
9. Handling cold start: new user → use demographics + context; new item → use content features only

Follow-ups:

- ”How do you handle cold start?” → Content-based features for new items; exploration strategies (ε-greedy, UCB) for new users
- ”How do you prevent filter bubbles?” → Diversity in re-ranking (MMR), inject exploration items, monitor content diversity index
- ”How would you handle feedback loops?” → Off-policy correction, causal inference, debiasing weights

Q2: "How do you detect and handle model degradation in production?"

Model answer:

1. **Detect**: Monitor three layers — data (input feature distributions via PSI), model (prediction distributions, confidence), business (CTR, revenue, latency)
2. **Alert thresholds**: PSI > 0.2 on key features; prediction mean shift > 2σ; business metric drops > X%
3. **Distinguish causes**:
   - Data pipeline issue → features are null/wrong → check upstream
   - Concept drift → retrain on recent data
   - Covariate shift → weighted retraining or domain adaptation
4. **Response playbook**: (a) alert on-call, (b) rollback to last stable model if severe, (c) investigate root cause, (d) retrain with corrected data, (e) validate before re-deployment
5. **Prevent future**: daily drift reports, automated retraining triggers, test data sets from multiple time windows

Follow-ups:

- ”Difference between data drift and concept drift?” → Data drift: P(X) changes, model may still be right if P(Y|X) unchanged; concept drift: P(Y|X) changes, requires retraining
- ”How do you get ground truth labels quickly?” → For ads: clicks are near-instant; for purchase predictions: 30-day delay; workaround: proxy labels (add-to-cart)

Q3: "Explain training-serving skew and how you prevent it."

Model answer: Training-serving skew occurs when the features computed at training time differ from features computed at serving time. This is the #1 production issue in ML systems.

Causes:

1. Different code paths for batch training vs. online serving
2. Using stale features at training time (e.g., features from 1 week before the event)
3. Data leakage: using future information in training features
4. Different preprocessing logic (normalize differently)

Prevention:

1. **Feature store**: single feature computation logic consumed by both training (via offline store) and serving (via online store)
2. **Log-and-join**: log features at serving time, join with labels later; training uses these exact served features
3. **Point-in-time correct joins**: in training data preparation, only use features that were available at prediction time
4. **Test for skew**: statistical comparison of feature distributions between training set and serving logs

Q4: "What's the difference between data parallelism and model parallelism?"

Model answer: Data parallelism: Every worker holds a full copy of the model. Data is split — each worker processes a different mini-batch. Workers compute gradients independently, then gradients are averaged (via all-reduce). Works when model fits on one GPU. Most common approach (PyTorch DDP, Horovod).

Model parallelism: Model is too large for one GPU. Model is split across workers — different layers on different GPUs. Forward pass flows activations from GPU to GPU through the pipeline. Backward pass flows gradients in reverse. Used for LLMs (GPT-4, Llama 70B).

Pipeline parallelism is a subset of model parallelism that addresses GPU idling: split mini-batches into micro-batches, pipeline them through stages so no GPU is idle waiting.

In practice: Use data parallelism first. Only use model parallelism when model > GPU memory.

Q5: "How do you run A/B tests for ranking systems?"

Model answer: Standard A/B tests are suboptimal for ranking because: they need large sample sizes (weeks), high variance, and network effects corrupt randomization.

Better approaches:

1. **Interleaving**: For each user, merge ranked lists from model A and model B (random tiebreak for equal positions). Observe which model's items get clicked more. 100x more sensitive than A/B, runs in days instead of weeks. Used by Netflix, Spotify.
2. **Quasi-experimental**: Use geographic or temporal holdouts if randomization is impossible (marketplace dynamics, social graphs).
3. **Multi-armed bandit**: For when you want to explore multiple variants while exploiting the current best (Thompson sampling, UCB).

For any experiment:

- Pre-register hypothesis and metric
- Calculate required sample size before starting (avoid peeking)
- Run for 2+ weeks to average over weekly cycles and overcome novelty effect
- Analyze primary metric + guardrail metrics + segment breakdowns

1.12 Senior-Level Discussion Points

1. Feedback loops and their dangers: When a model influences the data used to train its successor, feedback loops emerge. Example: recommendation model recommends popular items → popular items get more clicks → model trains on these skewed clicks → recommends even more popular content → ”rich get richer.” Solution: mix exploration (ε-greedy, Thompson sampling) with exploitation; off-policy evaluation; counterfactual logging.

2. Causal inference vs. correlation in ML systems: Standard ML learns correlation. For ads, correlation isn't enough — we need to know if showing ad X CAUSED a purchase, not just that purchases co-occurred with ad views. Approaches: randomized experiments (gold standard), instrumental variables, doubly-robust estimators, counterfactual reasoning.

3. Multi-objective optimization is a product decision, not a model decision: The choice of how to weight P(engagement) vs. P(wellbeing) vs. P(revenue) is a values-laden business decision, not a technical one. Senior engineers push back on framing it as a modeling problem and involve policy, legal, and product stakeholders.

4. MLOps maturity model:

- Level 0: Manual, ad-hoc training and deployment (notebook to production)
- Level 1: Automated training pipeline, manual deployment trigger
- Level 2: Automated training + automated evaluation + automated deployment (CT/CD)
- Level 3: Full experimentation platform with multi-arm bandits, online learning

5. The hidden technical debt in ML systems (D. Sculley et al., ”Machine Learning: The High-Interest Credit Card of Technical Debt”):

- Boundary erosion: ML models interact with everything but are tested in isolation
- Entanglement: changing one feature changes all features (CACE principle — Change Anything, Change Everything)
- Undeclared consumers: model outputs consumed by downstream systems without documentation
- Data dependencies: harder to track and eliminate than code dependencies

6. LLMOps — how LLMs change MLOps:

- **Prompt versioning**: prompts are code, must be version-controlled and tested
- **Eval harness**: automated evaluation against golden datasets before deployment
- **Guardrails**: output filtering for safety, hallucination detection
- **Observability**: token usage, latency, refusal rates, harmful output rates
- **Fine-tuning vs. RAG vs. prompt engineering**: build-vs-buy decision for each use case
- **Cost management**: LLM inference is expensive; caching, routing to smaller models for simple queries
- **Context management**: token limits require smart chunking, retrieval, and context compression

7. Fairness, accountability, and transparency:

- Slice-based evaluation: ensure model performs equally across demographic groups
- Disparate impact analysis: measure if model outcomes disproportionately affect protected groups
- Model cards: documentation of model capabilities, limitations, evaluation conditions
- Counterfactual fairness: if a protected attribute changed, would the prediction change?

1.13 Typical Mistakes Candidates Make

Mistake 1: Jumping to model architecture too quickly. *Problem*: Candidates start designing neural network architectures before clarifying requirements, defining metrics, or discussing data. *Fix*: Always spend the first 5 minutes on Step 1 (clarify) and Step 2 (objective/metrics). Say ”before I discuss models, let me make sure I understand the problem.”

Mistake 2: Ignoring training-serving skew. *Problem*: Describing a training pipeline and a serving pipeline independently without addressing how features will be consistent. *Fix*: Explicitly mention feature

stores and point-in-time correct joins. Say ”to prevent training-serving skew, I'd use a feature store where the same feature computation logic is used for both.”

Mistake 3: Forgetting monitoring and the feedback loop. *Problem*: The design ends at serving. No mention of how the system detects degradation or improves over time. *Fix*: Always complete the loop. End with monitoring (data drift, prediction drift, business metrics), alerting, and retraining triggers.

Mistake 4: Using wrong evaluation strategy. *Problem*: Using random train/test split on time-series interaction data. *Fix*: Always use temporal split for ML systems (train on past, evaluate on future). Mention this explicitly.

Mistake 5: Only discussing offline metrics. *Problem*: ”We'd optimize for AUC of 0.85.” No mention of online A/B tests. *Fix*: Distinguish offline evaluation (AUC, NDCG — fast iteration signal) from online evaluation (A/B test of business metric — ground truth). Both are required before shipping.

Mistake 6: Not handling cold start. *Problem*: Design assumes every user has rich history. Ignores new users and new items. *Fix*: Explicit cold start strategy: content-based features for new items; population-level statistics for new users; exploration vs. exploitation policies.

Mistake 7: Over-engineering the model. *Problem*: Proposing a transformer-based model when logistic regression on 5 features would solve the problem. *Fix*: Start with the simplest model that could work. Justify complexity with scale, data volume, and business requirement.

Mistake 8: Ignoring position bias and feedback bias. *Problem*: Using click data as-is without noting that items shown at position 1 always get more clicks. *Fix*: Mention position bias correction: ”I'd use inverse propensity scoring or train a position-aware model to debias click labels.”

Mistake 9: Not discussing latency budget. *Problem*: Describing a complex multi-stage pipeline without accounting for how each step affects total latency. *Fix*: For each stage, explicitly state: ”candidate retrieval < 20ms, feature serving < 10ms, ranking < 50ms, total < 100ms.”

Mistake 10: Conflating data drift and concept drift. *Problem*: Using both terms interchangeably. *Fix*: Data drift = input distribution P(X) changed. Concept drift = the relationship P(Y|X) changed. Both require different responses.

1.14 How This Connects To Other Topics

ML Fundamentals:

- Bias-variance tradeoff determines whether you need more data (high bias) or regularization (high variance)
- Feature selection directly impacts both online store size and serving latency
- Calibration (from probability theory) is critical for ad auction pricing

Deep Learning:

- Two-tower models use the same building blocks as standard neural networks
- Distributed training (all-reduce, model parallelism) is required for large deep learning models
- Embedding layers are the core of how categorical features are handled in production

System Design:

- Feature stores are databases with specific consistency and latency requirements
- Serving systems are distributed services with load balancing, caching, and fault tolerance
- Monitoring pipelines are stream processing systems (Kafka, Flink)

Distributed Systems:

- All-reduce algorithms for gradient aggregation borrow from consensus protocols
- Parameter servers are distributed key-value stores
- Feature stores require distributed storage (Redis cluster, Cassandra)

Cloud Infrastructure:

- Managed ML platforms (Vertex AI, SageMaker, Azure ML) implement much of the MLOps stack
- Spot instances / preemptible VMs dramatically reduce training costs for data parallel jobs

- Object storage (S3, GCS) is the foundation of offline feature stores and model registries

Statistics:

- A/B testing validity depends on correct hypothesis testing, p-values, power analysis
- Drift detection uses KL divergence, Kolmogorov-Smirnov test, chi-squared tests
- Causal inference methods (IV, DiD) are required for unbiased effect estimation

Data Engineering:

- Feature pipelines run on Spark/Flink/dbt — all standard data engineering tools
- Data quality tools (Great Expectations, Deequ) originate in data engineering
- Schema evolution and data versioning (Delta Lake) are data engineering concerns

1.15 FAANG Interview Tips

For Google / YouTube:

- Emphasize scale (billions of users, millions of QPS)
- Know TF Serving, TFX, Vertex AI well
- Be ready to discuss ad auction mechanics and CTR model calibration
- System design: expect Ads, YouTube recommendation, Search ranking

For Meta / Facebook:

- Know DLRM architecture (they open-sourced it)
- Be familiar with feed ranking, ads, Reels recommendation
- Discuss multi-objective optimization trade-offs between engagement and wellbeing
- Strong emphasis on A/B testing rigor — Meta runs thousands of experiments simultaneously

For Amazon:

- Product recommendation = core business driver
- E-commerce-specific challenges: price as a feature, inventory constraints, session length
- Know SageMaker ecosystem
- Discuss cold start for long-tail products (millions of products with few interactions)

For Netflix:

- Watch ”A/B Testing on Netflix” talks — interleaving is their secret weapon
- Two-tower model for candidate generation is well-documented in their tech blog
- Causal inference and counterfactual evaluation is a strong emphasis
- Know heterogeneous treatment effects (different users respond differently to recommendations)

For LinkedIn:

- Economic Graph embeddings — professional entity representations
- Feed ranking with ”dwell time” as primary positive signal
- Recommendation in professional context: be aware of fairness to recruiters, job seekers, applicants
- AI governance and fairness teams are prominent at LinkedIn

General FAANG tips:

- Draw the complete pipeline, not just the model
- Mention specific tools by name (don't just say ”a database” — say Redis for online store)
- Quantify: mention sample sizes, latency budgets, model complexity (number of params)
- Explicitly address the full lifecycle: train $\rightarrow$ deploy $\rightarrow$ monitor $\rightarrow$ retrain
- When asked ”how would you improve this?”, propose A/B testable hypotheses, not theoretical improvements
- Demonstrate that you've shipped production ML, not just trained notebooks

1.16 Revision Cheat Sheet

10-Minute Summary

MLOps solves the gap between model training and reliable production impact. The core loop is: **data** → **features** → **train** → **evaluate** → **serve** → **monitor** → **retrain**. Feature stores eliminate training-serving skew by providing a single feature computation layer for both offline training and online serving. Distributed training uses data parallelism (copy model, split data) for large datasets and model parallelism (split model across GPUs) for giant models. All-reduce (ring-based) is more bandwidth-efficient than parameter servers for homogeneous clusters. Serving choices are batch (hours, cheap), online (< 100ms, expensive), or streaming (seconds). A/B tests validate offline improvements translate to online business metrics; interleaving is 100x faster for ranking systems. Monitor for four drift types: data, concept, prediction, label. Detect with PSI (> 0.2 = act). The recommendation funnel is: candidate generation (millions → hundreds via ANN) → ranking (deep model) → re-ranking (diversity, business rules). CTR prediction requires calibrated probabilities (not just discrimination) because auction prices depend on exact predicted values.

Key Points

1. **Framework**: Clarify → Objective → Data → Features → Model → Train → Evaluate → Serve → Monitor → Iterate
2. **Training-serving skew prevention**: single feature computation logic, feature store, log-and-join
3. **Feature store**: offline (S3, point-in-time correct) + online (Redis, < 10ms) + shared computation logic
4. **Distributed training**: data parallel (same model, split data, all-reduce gradients) vs. model parallel (split model, flow activations)
5. **All-reduce**: ring topology, each GPU sends/receives exactly one full gradient, bandwidth-optimal
6. **Serving**: batch (offline, cheap) vs. online (< 100ms, GPU cluster) vs. streaming (Flink, seconds)
7. **Recommendation funnel**: Two-tower retrieval (ANN, < 10ms) → Neural ranking (Deep & Cross, < 100ms) → Re-ranking (diversity + business rules, < 50ms)
8. **Drift types**: data ($P(X)$ changes), concept ($P(Y|X)$ changes), prediction ($\hat{Y}$ distribution), label ($P(Y)$)
9. **PSI thresholds**: < 0.1 stable, 0.1-0.2 monitor, > 0.2 act
10. **A/B testing**: interleaving 100x faster for ranking; run 2+ weeks for novelty effect; correct for multiple testing
11. **CTR model**: must be calibrated (exact probabilities matter for auction); DLRM/Wide & Deep; extreme class imbalance
12. **LLMOps additions**: prompt versioning, eval harness, guardrails, cost management, RAG vs. fine-tune decision

Cheat Sheet Table

Concept	Key Detail	Interview Trigger
Feature store	Offline (S3) + Online (Redis) + shared code	”training-serving skew” question
Data parallelism	Full model per GPU, split data, all-reduce	”distributed training” question
Model parallelism	Split model, flow activations	”large model / LLM” question
Ring all-reduce	Each GPU sends receives one full tensor total	”why not parameter server?”
Two-tower model	User tower + item tower + dot product + ANN	”recommendation system”
Candidate → Rank → Re-rank	Millions → 500 → 50 → 20	”recommendation funnel”
Wide & Deep / DCN	Dense + sparse + cross features	”CTR prediction model”
PSI > 0.2	Major distribution shift, take action	”drift detection”

Concept	Key Detail	Interview Trigger
Concept drift	P(Y|X) changed, not just P(X)	”monitoring” question
Interleaving	100x faster than A/B for ranking	”online evaluation” question
Point-in-time join	No future leakage in training features	”training-serving skew”
Continuous training	Retrain triggered by drift or schedule	”retraining strategy”
Batch serving	Pre-compute, hours old OK, cheap	”latency not critical”
Online serving	Request-triggered, < 100ms, GPU cluster	”real-time scoring”
Calibration	P(click) exact value must be correct	”ad auction” question

Most Important Concepts

For the interview, these 5 concepts matter most:

1. **The full lifecycle** (data → features → train → serve → monitor → retrain) — draw this diagram, walk through it explicitly
2. **Feature store solving training-serving skew** — understand offline vs. online stores, point-in-time correctness, and why same feature code eliminates skew
3. **Recommendation funnel** (two-tower candidate generation → deep ranking → re-ranking) — know why each stage exists and what it trades off
4. **Drift detection and response** — four drift types, PSI metric, monitoring loop, automated retraining triggers
5. **A/B testing for ML systems** — offline vs. online evaluation gap, interleaving for ranking, statistical rigor requirements (sample size, multiple testing, novelty effect)

Word count: approximately 12,500 words | Section count: 16 sections